LAB SHEET - 10

Name: Shreya Aggarwal
Section: C
Roll No.: 58
CU-ID: CU24250082

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow version:", tf.__version__)
print("Libraries imported successfully")
```

```
TensorFlow version: 2.21.0
Libraries imported successfully
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

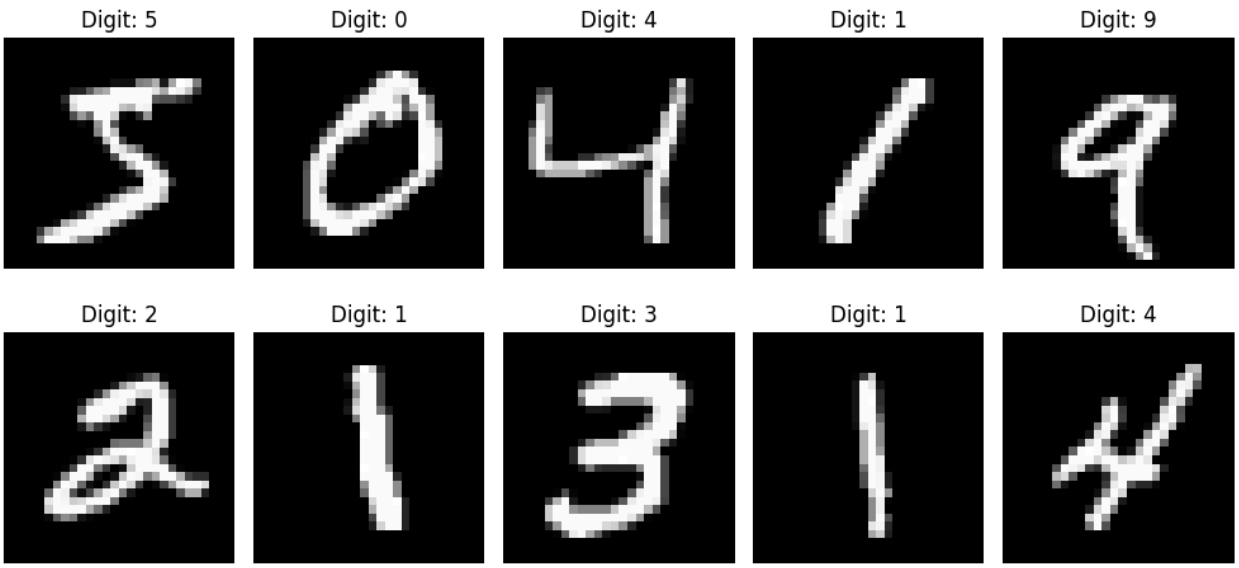
print("MNIST dataset loaded successfully")
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
MNIST dataset loaded successfully
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
```

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Digit: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)
```

```
print("Image preprocessing completed successfully")
print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
```

```
Image preprocessing completed successfully
Training data shape: (60000, 28, 28, 1)
Testing data shape: (10000, 28, 28, 1)
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("CNN model created and compiled successfully")
```

```
CNN model created and compiled successfully
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`inpu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(32, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("CNN model created and compiled successfully")
```

```
CNN model created and compiled successfully
```

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 128)	204,928
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)

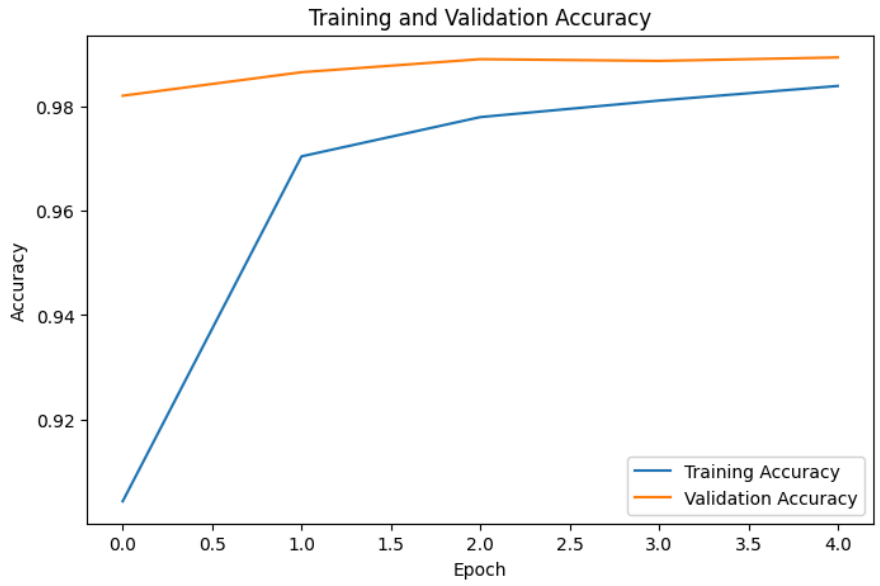
```
history = model.fit(
    x_train,
    y_train,
```

```
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)

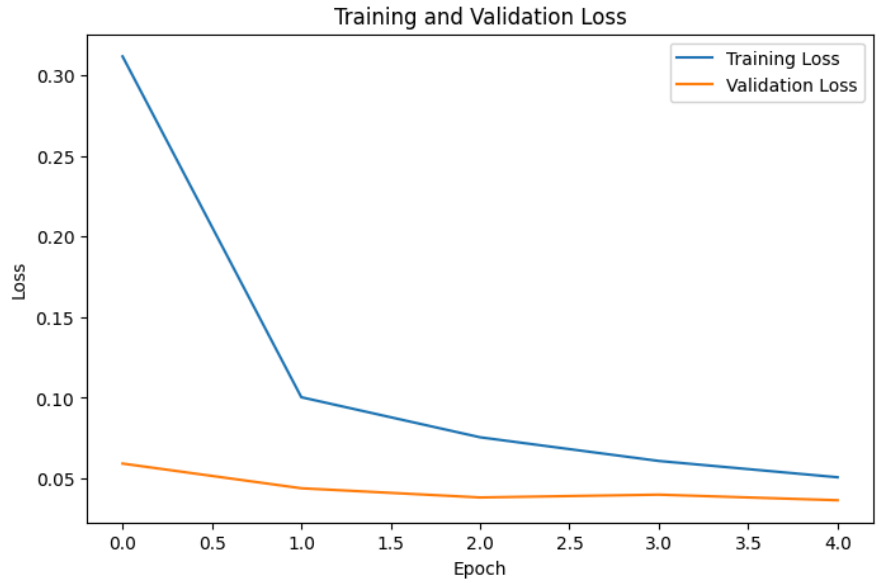
print("CNN training completed successfully")
```

```
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 43s 98ms/step - accuracy: 0.9044 - loss: 0.3118 - val_accuracy: 0.9820 - val_loss: 0.0592
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 40s 96ms/step - accuracy: 0.9704 - loss: 0.1004 - val_accuracy: 0.9865 - val_loss: 0.0439
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 41s 97ms/step - accuracy: 0.9779 - loss: 0.0756 - val_accuracy: 0.9890 - val_loss: 0.0382
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 40s 94ms/step - accuracy: 0.9811 - loss: 0.0609 - val_accuracy: 0.9887 - val_loss: 0.0400
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 41s 93ms/step - accuracy: 0.9839 - loss: 0.0507 - val_accuracy: 0.9893 - val_loss: 0.0365
CNN training completed successfully
```

```
plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.show()
```




```
plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.show()
```




```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=1
)

print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy, 4))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.9899 - loss: 0.0323
Test Loss: 0.0323
Test Accuracy: 0.9899
```

```
from sklearn.metrics import classification_report

y_pred = model.predict(x_test, verbose=0)
y_pred_classes = np.argmax(y_pred, axis=1)

report = classification_report(
    y_test,
    y_pred_classes,
    digits=4
)

print("Classification Report")
print(report)
```

```
Classification Report
              precision    recall  f1-score   support

           0     0.9898    0.9939    0.9919       980
           1     0.9912    0.9974    0.9943      1135
           2     0.9884    0.9903    0.9894      1032
           3     0.9891    0.9891    0.9891      1010
           4     0.9949    0.9980    0.9964       982
           5     0.9822    0.9899    0.9860       892
           6     0.9958    0.9885    0.9921       958
           7     0.9743    0.9951    0.9846      1028
           8     0.9979    0.9795    0.9886       974
           9     0.9960    0.9762    0.9860      1009

    accuracy                         0.9899     10000
   macro avg     0.9900    0.9898    0.9898     10000
weighted avg     0.9900    0.9899    0.9899     10000
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

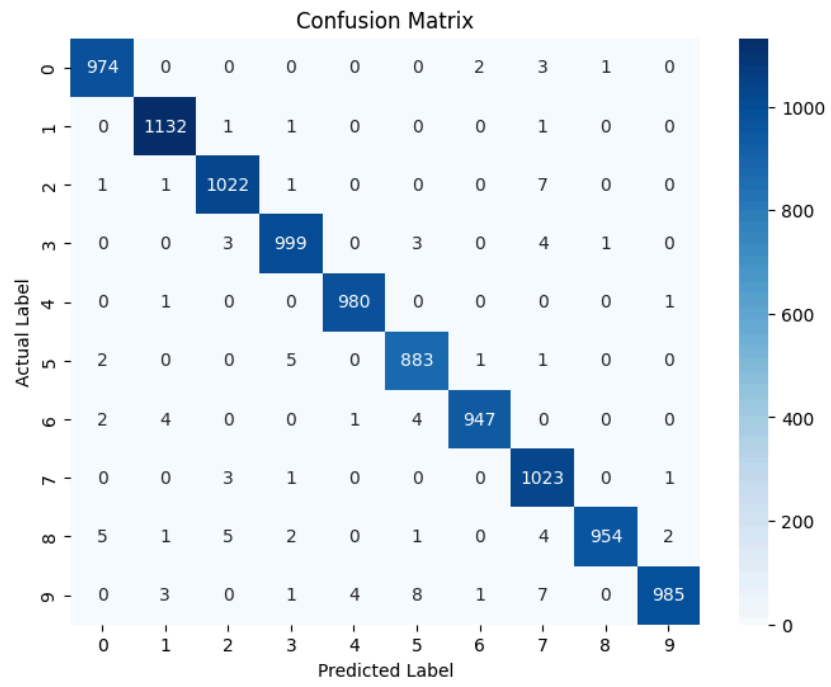
```
Start coding or generate with AI.
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
import seaborn as sns

cm = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=range(10),
    yticklabels=range(10)
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix")
plt.show()
```



Start coding or generate with AI.

Start coding or generate with AI.

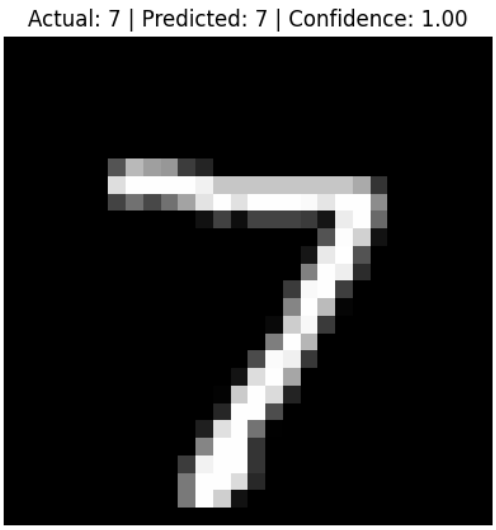
Start coding or generate with AI.

```
index = 0

prediction = model.predict(x_test[index:index+1], verbose=0)
predicted_label = np.argmax(prediction)
actual_label = y_test[index]
confidence = np.max(prediction)

plt.figure(figsize=(5, 5))
plt.imshow(x_test[index].squeeze(), cmap="gray")
plt.title(
    f"Actual: {actual_label} | "
    f"Predicted: {predicted_label} | "
    f"Confidence: {confidence:.2f}"
)
plt.axis("off")
plt.show()

print("Actual Label:", actual_label)
print("Predicted Label:", predicted_label)
print("Confidence:", round(float(confidence), 4))
```




```
Actual Label: 7
Predicted Label: 7
Confidence: 1.0
```

Summary

Dataset: MNIST Handwritten Digits
Model Used: Convolutional Neural Network (CNN)
Number of Classes: 10 (0–9)
Training Images: 60,000
Testing Images: 10,000
Epochs: 5
Optimizer: Adam
Test Accuracy: 98.99%
Test Loss: 0.0323
Sample Digit: 7
Actual Label: 7
Predicted Label: 7
Prediction Confidence: 1.00

Result

The CNN was successfully trained on the MNIST dataset for handwritten digit classification. The model achieved 98.99% accuracy on the test dataset. The sample test image was correctly classified as digit 7 with a confidence of 1.00. The training and validation graphs and